وثيقة المراجعة والتلخيص الأكاديمي

ASE · Chapter 5 · Software Design

المستوى: Create + Evaluate + Analyze + Apply — فوق مستوى أسئلة الكتاب. **ليش هذا الملف:** الدكتور يعتمد أسئلة السلايدات (سطحية). هذا الملف يغطي **الهامش اللي هو ما يشرحه** — وهنا الفرق بين النجاح والتفوق. **قاعدة:** كل جواب مُشتق من نص الفصل (سلايدات 1–99 + كتاب ص.202–251). ماكو مصدر خارجي.

المجموعة 1 — فهم (Understand / Comprehension)

Q1.1 Every type of coupling that involves *passing something* is bad; every type that involves *sharing something* is worse. Explain this pattern and why it holds. **Answer:** When modules only *pass* information (data, a data structure, a control flag), the dependency is one-directional and traceable — the caller knows what it sends. When modules *share* a resource (global data area in common coupling; another module's internals in content coupling), the dependency is two-directional and hidden — neither module can be understood alone, and a change in either one silently affects the other. Sharing removes the interface contract as the single point of control, so sharing is always worse than passing.

Q1.2 Why is *cohesion* defined inside a module and *coupling* defined between modules, and why must both be considered together? **Answer:** Cohesion is intra-module — it measures how strongly the elements *within* one module belong together. Coupling is inter-module — it measures how strongly two modules depend on each other. They must be considered together because they are two views of the same partition: if you split a module badly (low cohesion), the resulting pieces still need to cooperate, which forces extra inter-module links (high coupling). You cannot improve one by ignoring the other.

Q1.3 Explain the difference between "a module is easy to use" and "a module is easy to build", and which property the design should favour. **Answer:** Easy to use means the module presents a small, well-defined interface that hides its internal complexity — a caller needs to know only *what* it does. Easy to build means the internal implementation is simple from the implementer's view. The design should favour **easy to use** (the module must be simpler from outside than from inside), because usability drives reuse and lowers coupling, whereas an easy-to-build module with a messy interface raises coupling for every caller.

Q1.4 Why is hybrid design called the "most practical" strategy? **Answer:** Pure top-down cannot reuse existing/library modules and leaves low-level design decisions late; pure bottom-up risks building modules nobody needs and delays the overall architecture. Hybrid takes the strength of each: the top-down view defines the *major modules and overall structure* early, while the bottom-up view is applied near the *bottom of the hierarchy* (where there are more modules, intuition is simpler, and library reuse and bottom-up testing matter). Practically, real projects always mix the two.

Q1.5 Distinguish an object's "state" from its "behaviour", and explain how these map into a class diagram. **Answer:** State is the set of data/attribute values the object remembers; behaviour is the set of operations the object can perform, which change or examine that state. In a class diagram, state appears in the **attribute**

(middle) compartment and behaviour in the **operation (lower) compartment** — and all objects of the class share the same operations while each holds its own attribute values.

المجموعة 2 — تحليل (Analyze)

Q2.1 Worked classification: A module validate_and_save( ) reads a record from a global buffer, validates it, and writes it back to the same global buffer. Classify its coupling to the rest of the system and its internal cohesion. Justify each. **Answer:** Coupling = **Common coupling** (worst-but-one) — the module and others communicate through a shared *global data area*, which both read and write, so neither can be changed alone. Cohesion = **Sequential cohesion** (good) — the sequence is "read → validate → write", where the output of one step is the input of the next, so the elements are chained by data flow. Diagnosis: internal cohesion is fine, but the global buffer is the real defect; passing the record as a parameter instead would move it to the best coupling (data).

Q2.2 Worked classification: A module run_initializations( ) contains "open log file", "initialize device drivers", "set default time zone", and "clear temporary tables". Classify coupling and cohesion. **Answer:** Cohesion = **Temporal cohesion** — the elements are grouped only because they must all execute at the same time (system startup), not because they are functionally related. Coupling = likely **Common coupling** — initialisation routines typically write to shared global state (config, device tables), so they touch a shared data area.

Q2.3 Three modules A→B→C→D are chained so that B calls C and C calls D. Module A calls B. Module D is changed. Explain the ripple effect and how coupling level changes its severity. **Answer:** The ripple effect is the propagation of change from a modified module to its dependents. Because the chain is bidirectional in dependency (B depends on C, C on D), a change to D can force changes in C, then B, then A. If these links are data-coupled, only the D–C interface (parameter list) must be re-checked; if they are content-coupled, D's change can silently break C's internals, and the ripple reaches every dependent with no compiler help. Thus the *form* of coupling determines whether the ripple is contained or cascades.

Q2.4 A design has one giant "utility" module containing forty unrelated helper functions. Analyse the design's problems in terms of cohesion, coupling and the four design objectives. **Answer:** Cohesion = **Coincidental** (worst) — the functions have little or no relationship. Because everything calls the utility, coupling becomes **Common/Content-level** — many modules depend on the same unit, so any change ripples everywhere. Against the objectives: it is **not understandable** (nothing meaningful unifies it), **not maintainable** (every change is high-risk), **not correct/complete** in isolation, and it sits at the **wrong level** of abstraction. The fix is to split by function into cohesive modules with data-only interfaces.

Q2.5 Explain why logical cohesion is placed above coincidental cohesion, and why it is still far from functional cohesion. **Answer:** Logical cohesion contains elements that **appear related because they fall into the same logical class** (e.g. all input routines), which is a real, nameable relationship — so it is not random, and therefore better than coincidental. But it is still weak because the module performs *many different actual tasks* chosen at runtime (usually by a control flag), so the caller must pass control information (raising coupling to control level) and each task cannot be understood without reading the whole module.

Q2.6 The book presents both function-oriented and object-oriented design as "orthogonal". Analyse what orthogonality means here and give one problem each style naturally models well. **Answer:** Orthogonal means the two decompositions run along *different axes*: function-oriented decomposes by **what the system does** (process/function), while object-oriented decomposes by **the data/things the system manipulates** (objects).

They are perpendicular viewpoints of the same system. Function-oriented design naturally models **computational/algorithmic systems** (e.g. a numerical pipeline where the transformation steps dominate). Object-oriented design naturally models **real-world entity systems** (e.g. a library system of books, students and loans, where the things and their states dominate).

Q2.7 Analyse why "information hiding" (encapsulation) reduces coupling at the module level, not just at the object level. **Answer:** Encapsulation separates the external aspect of an entity from its internal implementation. This means the interface (the contract) is the *only* exposed surface, and the internal details cannot be depended on. Dependencies can therefore attach only to the stable interface, never to internals — which is exactly what distinguishes data coupling from content coupling. Hence the same principle that protects an object's data also protects a module's internals against content coupling.

Q2.8 In the Library Management System case study, why are separate use-case descriptions written for "Issue Book" and "Return Book" instead of one "Circulation" use case, when analysed by cohesion principles? **Answer:** Because the two flows have different actors, pre-conditions, post-conditions and alternate flows. Merging them into one "Circulation" use case would create a **logically-cohesive** artefact: one description whose behaviour branches on a control condition (issue vs return) — the model-level equivalent of control coupling in code. Keeping them separate gives each a single, well-defined purpose (functional cohesion), making the model easier to read, verify and implement.

المجموعة 3 — تقييم (Evaluate)

Q3.1 Design A: 12 small modules, data-coupled, each functionally cohesive, but with 40 inter-module calls. Design B: 5 large modules, common-coupled, logically cohesive, with 12 inter-module calls. Which is the better design, and why? Under what condition would B be defensible? **Answer: Design A is better.** Functional independence is the goal — high cohesion plus low coupling — and A achieves both, even though the *number* of calls is higher. Number of calls is not a defect if each call is a clean data-coupled interface; B's low call-count is bought with common coupling and logical cohesion, which means a change ripples across the shared globals and no module can be understood alone. **B is defensible only** in a hard real-time system where the 40 calls' overhead is genuinely unacceptable *and* the shared state is a single well-controlled resource — and even then it should be documented as a deliberate trade-off, not presented as good design.

Q3.2 Your team must deliver a system where the requirements are expected to change heavily over time. Evaluate whether to choose function-oriented or object-oriented design, with justification. **Answer:** Choose **object-oriented design**. The book's stated reason for OOD's improvement is that it begins with the real-world things of the problem, which are *stable* compared to functions: requirements change by adding or altering operations on entities far more often than by replacing the entities themselves. Encapsulation localises the change to the affected class, and inheritance/polymorphism let new behaviour be added by subclassing rather than by rewriting — both directly attacking the ripple effect. Function-oriented design would spread a single new requirement across several function hierarchies.

Q3.3 A student argues: "Since content coupling is the worst, and data coupling is the best, I should never pass a data structure — only primitives." Evaluate. **Answer:** The premise is right, the conclusion is over-strict. Passing a whole structure is **stamp coupling**, which *is* worse than data coupling because the called module gains access to more than it needs — but it is still far better than control, external, common or content coupling. Refusing all

structured parameters would force the caller to unpack every field into primitives, which **raises** the number of parameters and can create control coupling when flags are added. The correct rule is: pass **the minimum data required**, prefer primitives when natural, and accept a structure when it genuinely represents one datum — never pass more than the module needs.

Q3.4 Evaluate the slide claim that "modularity is the single attribute of software that allows a program to be intellectually manageable." Give the strongest argument for and the strongest criticism of this claim. **Answer: For:** without module boundaries the human mind cannot hold a whole system in working memory; modularity is what partitions the problem, and the properties it brings (separately compilable, simple interface, well-defined purpose) are the preconditions for every other quality (understandability, maintainability, reuse). **Against:** modularity alone is insufficient — a badly modularised system (coincidental cohesion + common coupling) is *harder* to manage than a small unmoduleised one, and the claim's own chapter shows that the *quality* of the modularisation (coupling/cohesion) is what delivers manageability. The stronger claim is: "modularity is necessary but not sufficient — functional independence is the true attribute."

Q3.5 Rank the following four defects by severity and justify the order: (i) a module with temporal cohesion; (ii) a stamp-coupled pair; (iii) a content-coupled pair; (iv) a module with logical cohesion. **Answer:** Most severe → least: **(iii) content coupling** (breaks encapsulation, invisible change, untraceable), then **(iv) logical cohesion** (forces control coupling on every caller and cannot be named or tested meaningfully), then **(i) temporal cohesion** (weak but honest — the elements at least execute together for a real reason), then **(ii) stamp coupling** (a mild over-exposure; the module still receives data, just more than needed). The order follows the principle that *hidden, bidirectional* defects are worse than *visible, one-directional* ones.

المجموعة 4 — تصميم (Create)

Q4.1 Given the requirements — "The system must register a student, record a course, enrol a student in a course, and produce a transcript" — design the module decomposition and classify the cohesion of each module you propose. **Answer (model design):** - register_student(student_data) — creates and stores a student record. **Functional cohesion** (single task). - record_course(course_data) — creates and stores a course record. **Functional cohesion**. - enrol_student(student_id, course_id) — validates both exist, then links them. **Functional cohesion** (one task: create the enrolment link). - generate_transcript(student_id) — reads the student's enrolments, fetches grades, formats output. **Sequential cohesion** (fetch → assemble → format, each step feeds the next). - database_layer — accessed only through data-coupled parameterised calls; no global record store (avoids common coupling). Diagnosis: levels range Functional → Sequential (good); coupling is data-only (best); no shared global state.

Q4.2 Design the class model for a "Parking Lot" system with at least three classes, and identify one relationship of each kind used in the Chapter 5 OOD notation. **Answer (model design):** Classes: Vehicle (attributes: plate, type; operations: enter(), exit()), ParkingSpot (attributes: id, occupied; operations: assign(), release()), Ticket (attributes: issueTime, fee; operations: calculate()). Relationships: ParkingSpot **aggregates** Vehicle (aggregation — a spot contains a vehicle while parked); Ticket **depends on** ParkingSpot and Vehicle (dependency — it needs their data to compute the fee); a Car/Truck/Motorcycle **generalises** Vehicle (generalisation — subclasses inherit state/behaviour, and calculateFee() is a polymorphic operation overridden per type).

Q4.3 Propose a simplified design process you would actually follow for a small project, and justify each step's necessity using the chapter's concepts. **Answer (model process):** 1) **Create the use case model** — establishes actors and system boundary (prevents scope creep, which is the model-level cause of coincidental cohesion). 2) **Draw the activity diagram** (if required) — captures dynamic flow, exposing ordering constraints (protects against mis-ordered procedural steps). 3) **Draw the interaction diagram** — shows messages between objects, i.e. the intended coupling, allowing it to be reviewed *before* it is coded. 4) **Draw the class diagram** — fixes the data and behaviour (state + operations) with visible relationships. 5) **Draw state chart / component & deployment diagrams** as needed — state transitions and physical distribution. 6) **Complete the design description** per IEEE STD 1016. Justification: each earlier step constrains the next so that design errors are caught while they are still cheap — the whole point of producing a design before coding.

Q4.4 Write the SDD organisation (Table 1 / Table 2 views) for the Library Management System, choosing concrete artefacts for each design view. **Answer (model): Decomposition description** → hierarchical decomposition diagram + natural-language module list (e.g. the issue-book module decomposed into validate / locate / update). **Dependency description** → structure chart showing how the library modules call each other and what data passes. **Interface description** → parameter tables / interface files listing every module's parameters and return types. **Detail description** → PDL/pseudocode of the internal algorithm of each entity (e.g. the fine-calculation routine). Entity attributes (identification, type, purpose, function, subordinates, dependencies, interface, resources, processing, data) are recorded per design entity throughout.

Q4.5 A colleague says: "Let's skip design and code directly — we'll refactor later." Argue against this using at least four distinct concepts from Chapter 5. **Answer (model rebuttal):** (1) **Design vs coding** — design is language-independent structure; coding is only its translation, so "coding directly" is really "designing accidentally, while writing code", with no review point. (2) **Modularity** — without deliberate module boundaries there is no intellectual manageability; refactoring cannot invent a decomposition that was never planned. (3) **Coupling/cohesion** — ad-hoc coding almost always produces content/common coupling and coincidental cohesion, which are the *most expensive* defects to refactor later. (4) **Design objectives** — the design must be correct, complete, understandable, at the right level and maintainable; none of these can be certified without an SDD. (5) **SDD as blueprint** — the SDD is the blueprint for implementation, planning and decision-making; skipping it removes the reference against which correctness can be judged.

المجموعة 5 — حساب وتطبيق (Apply / Calculation)

Q5.1 Modules: M1 calls M2, M3, M4. M2 calls M5. M3 calls M5, M6. M4 calls M6. Compute the **fan-out** of each module and identify the **fan-in** of M5 and M6. **Answer:** Fan-out (number of modules a module calls): M1 = 3 (M2,M3,M4); M2 = 1 (M5); M3 = 2 (M5,M6); M4 = 1 (M6); M5 = 0; M6 = 0. Fan-in (number of modules that call a module): **M5 = 2** (called by M2 and M3); **M6 = 2** (called by M3 and M4). Interpretation: M5 and M6 are the high-fan-in leaves — they are the reused, general-purpose modules and must be the most carefully designed (they carry the highest ripple risk).

Q5.2 Rank the following six coupling types from best to worst and state the single criterion that orders them: Data, Content, Control, Common, Stamp, External. **Answer:** Best → worst: **Data → Stamp → Control → External → Common → Content.** Criterion: the *nature and visibility of what is shared* — from passing the minimum primitive data (best), through passing more than needed (stamp) or passing control (control), to being tied to an external format (external), to sharing a global area (common), to reaching into another's internals

(content, worst). The less a module needs to know about another and the more one-directional the dependency, the better.

Q5.3 A module contains A, B and C where B takes A's output and C takes B's output, and all three also read a shared configuration table. State its cohesion and its coupling, and give the cheapest single change that improves the design. **Answer:** Cohesion = **Sequential** (A→B→C is a data-flow chain, good). Coupling = **Common** (all three read the shared configuration table). Cheapest effective change: **pass the required configuration values into the module as parameters** instead of reading the global table — this converts common coupling to data coupling without touching the internal A→B→C sequence, so the good cohesion is preserved while the worst remaining defect is removed.

Q5.4 Given the coupling best → worst scale D>S>C>E>Co>Ct and the cohesion best → worst scale F>Se>Com>P>T>L>Coinc, score a design consisting of three sequentially-cohesive, data-coupled modules and one coincidentally-cohesive, content-coupled module. Which single module should be refactored first? **Answer:** Scored by level: the three sequential/data modules sit near the best end (cohesion 2/7 from best; coupling 1/6 from best) — healthy. The single bad module sits at the worst end of **both** scales (cohesion 7/7, coupling 6/6). It must be refactored **first**, because it is simultaneously the worst on both axes: splitting it by function raises its cohesion, and replacing its content access with data parameters removes the highest-risk coupling. Refactoring any of the three good modules first would spend effort for almost no risk reduction.

خريطة التغطية (لا يوجد سؤال بلا مصدر)

المحور	Tier1	Tier2
Design definition / conceptual vs technical	5.1, 5.2	Q3.1, Q4.5
Design objectives / 4 properties	5.2	Q2.4, Q3.4
Modularity / properties	5.4	Q1.3, Q3.4
Coupling (types + order)	5.5, 5.8, 5.12	Q1.1, Q2.1, Q3.3, Q5.2
Cohesion (types + order)	5.6, 5.7, 5.9, 5.10	Q2.2, Q2.5, Q5.3
Relationship C&C	5.7	Q1.2, Q5.4
Strategy of design	5.11, 5.12	Q1.4
Function-oriented / notations	5.13, 5.14, 5.15	Q2.6, Q4.5
SDD / IEEE 1016	5.23	Q4.4
OOD concepts	5.16, 5.17	Q1.5, Q2.7
Inheritance/polymorphism	5.18, 5.19	Q4.2
OO vs FO design	5.22	Q2.6, Q3.2
Design process / case study	5.21, 5.20	Q2.8, Q4.1, Q4.3
Ripple effect	5.9, 5.10	Q2.3, Q5.1

المصدر: Singh & Aggarwal, Software Engineering 3rd ed., Ch.5 — سلايدات 1–99 + كتاب ص.202–251. كل جواب مُتحقق منه بالنص.

Retrieval set (active recall)

السؤال	الجواب
ترتيب الاقتران من الأفضل للأسوأ؟	Data > Stamp > Control > External > Common > Content
ترتيب التلاحم من الأفضل للأسوأ؟	Functional > Sequential > Communicational > Procedural > Temporal > Logical > Coincidental
كم خطوة OOD وشنو ترتيبها؟	6 — use case · activity · interaction · class · state · component/deployment
شنو SDD؟	مستند تصميم البرمجية — IEEE 1016
خصائص Vitruvius الثلاث؟	Durability · Utility · Charm
شنو الهدف الأعلى للتصميم؟	low coupling + high cohesion (سلايد 31)